

Python Programming

Assignment 1

Name: Pankaj Jadhav
Roll Number: MS2509

2nd August 2026

1 Variables

What is it?

A variable is a name that acts as a label for a value stored in the computer's memory. In Python, a variable is created the moment a value is assigned to it using the `=` operator, e.g. `x = 5`. Unlike many other languages, Python variables do not need their type to be declared in advance – the type is inferred automatically from the value assigned, and can even change if a new value of a different type is assigned later.

Why do we need it?

Variables let us store data temporarily so that it can be reused, updated, and referred to by a meaningful name instead of typing the raw value again and again. They make programs readable (`price` is easier to understand than `499`) and allow values to change as a program runs, which is essential for any kind of computation, looping, or user interaction.

Where and How is it Used?

Variables are used everywhere in a program: storing user input, counters in loops, results of calculations, configuration values, and so on. They are created simply by assignment:

```
1 name = "Pankaj"  
2 age = 23  
3 pi = 3.14159
```

Python also supports multiple assignment (`a, b = 1, 2`) and re-assignment of a different type to the same name, since variables are just references to objects.

2 Basic Data Types

2.1 Integer

What is it?

An integer (`int`) is a whole number, positive, negative, or zero, without a decimal point, e.g. `10`, `-3`, `0`. Python's integers have arbitrary precision, meaning they can grow as large as memory allows without overflowing.

Why do we need it?

Integers are needed to represent countable, discrete quantities such as the number of students, loop counters, indices, or any value where fractional parts don't make sense.

Where and How is it Used?

Used for counting, indexing lists/strings, loop control, and arithmetic that doesn't need fractional precision.

```
1 count = 5  
2 total_students = 60
```

2.2 Float

What is it?

A float (`float`) is a number with a decimal point, used to represent real numbers such as `3.14` or `-0.5`. Internally it is stored using floating-point representation, which means it can sometimes have small rounding errors.

Why do we need it?

We need floats whenever a quantity can take fractional values – prices, measurements, averages, scientific calculations, etc., where an integer alone is not precise enough.

Where and How is it Used?

Used in scientific computing, financial calculations, and any place where precision beyond whole numbers matters.

```
1 height = 5.9
2 average = 78.25
```

2.3 Boolean

What is it?

A boolean (`bool`) represents one of exactly two values: `True` or `False`. In Python, booleans are technically a subtype of integers, where `True` behaves like `1` and `False` behaves like `0`.

Why do we need it?

Booleans are essential for decision-making in programs. They represent the outcome of a condition or comparison and are what drive `if` statements, loops, and logical operations.

Where and How is it Used?

Used in conditional statements, loop conditions, and flags that track a program's state.

```
1 is_raining = True
2 is_passed = (marks >= 40)
```

2.4 String

What is it?

A string (`str`) is a sequence of characters enclosed in single, double, or triple quotes, used to represent text, e.g. `"Hello"`. Strings in Python are immutable, meaning once created they cannot be changed in place.

Why do we need it?

Strings are needed to store and manipulate textual data – names, messages, file contents, user input, and so on. Almost every program deals with text in some form.

Where and How is it Used?

Used for displaying messages, reading user input, processing text files, and building formatted output.

```
1 name = "Pankaj"
2 message = 'Welcome to python programming world!'
```

3 Arithmetic Operators

What is it?

Arithmetic operators are symbols used to perform mathematical operations on numeric values: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division, always returns a float), `//` (floor division), `%` (modulus / remainder), and `**` (exponentiation).

Why do we need it?

Almost every program needs to perform calculations – computing totals, averages, percentages, or checking divisibility. Arithmetic operators provide the basic building blocks for numerical computation.

Where and How is it Used?

Used in calculators, billing systems, scientific computations, and even in loop logic (e.g. using % to check if a number is even).

```
1 a, b = 7, 2
2 print(a + b)    # 9
3 print(a / b)    # 3.5
4 print(a // b)   # 3
5 print(a % b)    # 1
6 print(a ** b)   # 49
```

4 Comparison Operators

What is it?

Comparison operators compare two values and return a boolean result. They include == (equal to), != (not equal to), >, <, >=, and <=.

Why do we need it?

They allow a program to make decisions by testing relationships between values, which is the foundation of conditional logic (if/else) and loop termination conditions.

Where and How is it Used?

Used inside if statements, while loop conditions, and filtering operations.

```
1 x, y = 10, 20
2 print(x == y)   # False
3 print(x < y)    # True
```

5 Assignment Operators

What is it?

Assignment operators assign values to variables. Besides the plain =, Python provides compound assignment operators like +=, -=, *=, /=, //=, %=, and **= that update a variable's value based on its current value in a shorter form.

Why do we need it?

Compound assignment operators make code shorter and more readable when a variable needs to be updated based on its own previous value, such as incrementing a counter.

Where and How is it Used?

Very common in loops, accumulators, and running totals.

```
1 total = 0
2 total += 5    # same as total = total + 5
3 total *= 2    # same as total = total * 2
4 print(total) # 10
```

6 Type Conversion

What is it?

Type conversion (or type casting) is the process of converting a value from one data type to another, e.g. converting a string "5" into an integer 5 using int(). Python supports both implicit conversion

(done automatically by the interpreter, e.g. `int` to `float` in mixed arithmetic) and explicit conversion using functions like `int()`, `float()`, `str()`, and `bool()`.

Why do we need it?

Data does not always arrive in the type we need. For example, `input()` always returns a string, so if we need to do arithmetic on it, we must explicitly convert it to a number. Type conversion bridges these mismatches.

Where and How is it Used?

Used when processing user input, reading data from files, or combining values of different types in expressions.

```
1 age_str = "25"
2 age_int = int(age_str)
3 print(age_int + 5)  # 30
```

7 Input and Output

7.1 `input()`

What is it?

`input()` is a built-in function that pauses program execution and waits for the user to type something on the keyboard, returning whatever was typed as a string.

Why do we need it?

It allows a program to be interactive – taking data from the user at run time instead of having every value hard-coded in advance.

Where and How is it Used?

Used to collect names, numbers, choices, or any data from the user during execution.

```
1 name = input("Enter your name: ")
```

7.2 `print()`

What is it?

`print()` is a built-in function that displays output (text, numbers, variables) to the console/screen.

Why do we need it?

Without a way to display results, a program's computations would be invisible to the user. `print()` is the primary way Python communicates output back to us.

Where and How is it Used?

Used to show results, debug values, and format messages, including combining multiple values or using f-strings for formatting.

```
1 name = "Alice"
2 print("Hello,", name)
3 print(f"Hello, {name}!")
```

8 Strings

8.1 String Creation

What is it?

Strings can be created using single quotes (`'...'`), double quotes (`"..."`), or triple quotes (`'''...'''` / `"""..."""`) for multi-line text.

Why do we need it?

Different quoting styles let us conveniently include quote characters inside strings (e.g. use double quotes for a string that itself contains an apostrophe) and write long, multi-line text blocks cleanly.

Where and How is it Used?

```
1 s1 = 'Hello'
2 s2 = "World"
3 s3 = """This is
4 a multi-line string."""
```

8.2 Indexing

What is it?

Indexing refers to accessing an individual character of a string using its position number, starting at 0 for the first character. Negative indices count from the end, with -1 referring to the last character.

Why do we need it?

Indexing lets us pick out specific characters from text, which is essential for parsing, validation, and text processing tasks.

Where and How is it Used?

```
1 s = "Python"
2 print(s[0])    # 'P'
3 print(s[-1])   # 'n'
```

8.3 Slicing

What is it?

Slicing extracts a sub-portion (substring) of a string using the syntax `s[start:stop:step]`, where `start` is inclusive and `stop` is exclusive.

Why do we need it?

Slicing allows extraction of multiple characters at once (e.g. the first five characters, or every alternate character) without writing a loop, and is a very common and efficient tool in text processing.

Where and How is it Used?

```
1 s = "Programming"
2 print(s[0:5])    # 'Progr'
3 print(s[::-1])   # reversed string
```

8.4 String Operations

What is it?

String operations include concatenation (+), repetition (*), membership testing (in), length (len()), and built-in methods like `upper()`, `lower()`, `strip()`, `replace()`, and `split()`.

Why do we need it?

These operations let us build, clean, transform, and analyze text data, which is central to almost every real-world application (validating input, formatting output, parsing files).

Where and How is it Used?

```
1 greeting = "Hello" + " " + "World"
2 repeated = "=" * 10
3 print("Python" in greeting)  # False
4 print(greeting.upper())
```

9 Lists

9.1 Creating Lists

What is it?

A list is an ordered, mutable collection of items enclosed in square brackets `[]`, which can hold items of different types together, e.g. `[1, "two", 3.0]`.

Why do we need it?

Lists let us group related data together and work with a whole collection at once, instead of creating a separate variable for every single value.

Where and How is it Used?

```
1 numbers = [10, 20, 30]
2 mixed = [1, "apple", 3.5, True]
```

9.2 Indexing

What is it?

Just like strings, list elements can be accessed by their position, starting from 0, with negative indices counting from the end.

Why do we need it?

Indexing lets us retrieve or work with one specific element of a list at a time.

Where and How is it Used?

```
1 fruits = ["Apple", "Banana", "Mango"]
2 print(fruits[0])    # 'Apple'
3 print(fruits[-1])   # 'Mango'
```

9.3 Slicing

What is it?

Slicing a list, like a string, extracts a sub-list using `list[start:stop:step]`.

Why do we need it?

It allows extraction of a contiguous group of elements without looping manually.

Where and How is it Used?

```
1 fruits = ["Apple", "Banana", "Mango", "Orange"]
2 print(fruits[1:3])  # ['Banana', 'Mango']
```

9.4 Updating Elements

What is it?

Because lists are mutable, an element at a given index can be reassigned directly, e.g. `list[1] = 200`.

Why do we need it?

This lets us modify data in place – correcting a value, updating a score, etc. – without creating an entirely new list.

Where and How is it Used?

```
1 numbers = [10, 20, 30]
2 numbers[1] = 200
3 print(numbers)  # [10, 200, 30]
```

9.5 Nested Lists

What is it?

A nested list is a list that contains other lists as its elements, e.g. `[[1, 2], [3, 4]]`. This is how Python naturally represents 2D structures like matrices or grids.

Why do we need it?

Nested lists allow us to represent multi-dimensional or hierarchical data, such as a table, a matrix, or a list of student records.

Where and How is it Used?

```
1 matrix = [[1, 2, 3], [4, 5, 6]]
2 print(matrix[1][2])    # 6
```

9.6 Basic List Methods

What is it?

Lists provide built-in methods to modify or query them:

- `append(x)` – adds `x` to the end of the list.
- `insert(i, x)` – inserts `x` at index `i`.
- `remove(x)` – removes the first occurrence of value `x`.
- `pop(i)` – removes and returns the element at index `i` (last element if no index given).
- `sort()` – sorts the list in place, ascending by default.
- `reverse()` – reverses the order of elements in place.
- `len(list)` – returns the number of elements (technically a built-in function, not a method).

Why do we need it?

These methods provide ready-made, efficient ways to modify a list's contents and structure without writing manual loops for common tasks.

Where and How is it Used?

```
1 nums = [3, 1, 2]
2 nums.append(4)      # [3, 1, 2, 4]
3 nums.insert(0, 0)   # [0, 3, 1, 2, 4]
4 nums.remove(3)      # [0, 1, 2, 4]
5 nums.pop()          # removes 4 -> [0, 1, 2]
6 nums.sort()         # [0, 1, 2]
7 nums.reverse()      # [2, 1, 0]
8 print(len(nums))    # 3
```

10 Mini Experiments

Each experiment below was written and executed in Jupyter Notebook. The code, the actual output obtained, and an explanation are given for each.

10.1 Experiment 1: Swapping Variables

```
1 a = 10
2 b = 20
3 a, b = b, a
4 print(a)
5 print(b)
```

Output:

```
1 20
2 10
```

Explanation

Python evaluates the right-hand side (`b, a`) completely first, creating a temporary pair `(20, 10)`, before assigning it back to `(a, b)`. This is Python's built-in tuple-unpacking mechanism, which lets us swap two variables in a single line without needing a separate temporary variable, unlike in many other languages.

10.2 Experiment 2: String Indexing and Slicing

```
1 text = "Python Programming"
2 print(text[0])
3 print(text[-1])
4 print(text[0:6])
5 print(text[::-1])
```

Output:

```
1 P
2 g
3 Python
4 gnimmargorP nohtyP
```

Explanation

`text[0]` gives the first character, `'P'`. `text[-1]` gives the last character by counting from the end, `'g'`. `text[0:6]` slices out the first six characters (index 0 up to, but not including, index 6), giving `"Python"`. `text[::-1]` uses a step of `-1`, which walks through the string backward, reversing it entirely.

10.3 Experiment 3: List `append()` and `remove()`

```
1 numbers = [10, 20, 30]
2 numbers.append(40)
3 numbers.remove(20)
4 print(numbers)
```

Output:

```
1 [10, 30, 40]
```

Explanation

`append(40)` adds 40 to the end of the list, giving `[10, 20, 30, 40]`. `remove(20)` then searches for the first occurrence of the value 20 and deletes it, leaving `[10, 30, 40]`. Note that `remove()` deletes by *value*, not by index.

10.4 Experiment 4: More String Slicing

```
1 text = "Programming"
2 print(text[0:5])
3 print(text[3:])
4 print(text[:6])
5 print(text[::-1])
```

Output:

```
1 Progr
2 gramming
3 Progra
4 gnimmargorP
```

Explanation

`text[0:5]` takes characters at index 0-4 – "Progr". `text[3:]` omits the stop index, so it takes everything from index 3 to the end – "gramming". `text[:6]` omits the start index, so it takes everything from the beginning up to (not including) index 6 – "Progra". `text[::-1]` reverses the whole string as before.

10.5 Experiment 5: `input()` vs `int(input())`

```
1 age = input("Enter your age: ")
2 print(age + age)
3 age = int(age)
4 print(age + age)
```

Sample run (input: 21):

```
1 Enter your age: 21
2 2121
3 42
```

Why are the two outputs different?

`input()` always returns a **string**, even if the user types digits. So initially `age` holds the string "21", and `age + age` performs *string concatenation*, joining the two strings together to give "2121". Once we convert it using `int(age)`, `age` becomes the integer 21, and `age + age` now performs *numeric addition*, giving 42. The `+` operator behaves differently depending on the type of its operands – this is called operator overloading.

10.6 Experiment 6: List Indexing and Slicing

```
1 fruits = ["Apple", "Banana", "Mango", "Orange"]
2 print(fruits[0])
3 print(fruits[-1])
4 print(fruits[1:3])
```

Output:

```
1 Apple
2 Orange
3 ['Banana', 'Mango']
```

Explanation

`fruits[0]` is the first element, "Apple". `fruits[-1]` is the last element, "Orange". `fruits[1:3]` slices out elements at index 1 and 2 (index 3 is excluded), returning ["Banana", "Mango"] as a new list.

10.7 Experiment 7: Modifying a List Element

```
1 numbers = [10, 20, 30]
2 numbers[1] = 200
3 print(numbers)
```

Output:

```
1 [10, 200, 30]
```

Why can a list be modified while a string cannot?

Lists are **mutable** objects in Python, meaning their contents can be changed after creation – individual elements can be reassigned, added, or removed in place. Strings are **immutable**: once a string object is created, its characters cannot be changed individually (e.g. `"Python"[0] = "J"` raises a `TypeError`). To "modify" a string, Python must create an entirely new string object. This mutability/immutability distinction is a fundamental design choice: it makes strings safe to share and hash (e.g. as dictionary keys), while lists remain flexible for data that changes over time.

10.8 Experiment 8: Formatted Banner Output

```
1 print("=" * 40)
2 print("WELCOME")
3 print("=" * 40)
4 print()
5 print("*" * 20)
6 print("Python")
7 print("*" * 20)
```

Output:

```
1 =====
2 WELCOME
3 =====
4
5 *****
6 Python
7 *****
```

Explanation

The `*` operator, when used between a string and an integer, repeats the string that many times (string repetition). `"=" * 40` produces a line of 40 equal signs, and `"*" * 20` produces a line of 20 asterisks, which is a simple way to build banner-style decorative output. The empty `print()` simply prints a blank line as a separator.

10.9 Experiment 8 (continued): Reversing an Input Name

```
1 name = input("Enter your Name: ")
2 print(name[::-1])
```

Sample run (input: Alice):

```
1 Enter your Name: Alice
2 ecilA
```

Explanation

The slice `::-1` takes the whole string with a step of `-1`, i.e. it walks from the last character to the first, producing the reversed string. This is a common, concise Python idiom for reversing any sequence.

10.10 Experiment 9: Student Card

```

1 name = input("Enter Name: ")
2 age = input("Enter Age: ")
3 roll = input("Enter Roll Number: ")
4 course = input("Enter Course: ")
5
6 print("=====")
7 print("STUDENT CARD")
8 print("=====")
9 print("Name:", name)
10 print("Age:", age)
11 print("Roll No.:", roll)
12 print("Course:", course)
13 print("=====")
14 print()
15 print(f"Hello {name}!")
16 print("Welcome to the world of Python.")
17 print('Python is easy to learn!')
18 print("'I'll practice every day.'")
19 print("Practice makes perfect.")
20 print(r"\\\/\\\/\\\/\\\/\\\/\\\/\\\/\\\/\\\/")
21 print("Happy Coding!")

```

Sample run (inputs: Alice, 22, MS24001, M.Sc. Scientific Computing):

```

1 =====
2 STUDENT CARD
3 =====
4 Name: Alice
5 Age: 22
6 Roll No.: MS24001
7 Course: M.Sc. Scientific Computing
8 =====
9
10 Hello Alice!
11 Welcome to the world of Python.
12 "Python is easy to learn!"
13 'I'll practice every day.'
14 "Practice makes perfect."
15 \\\/\\\/\\\/\\\/\\\/\\\/\\\/\\\/\\\/
16 Happy Coding!

```

Explanation

Four separate calls to `input()` collect the name, age, roll number, and course as strings. The `print()` calls with comma-separated arguments automatically insert a single space between values, while the f-string `f"Hello {name}!"` embeds the variable directly inside the text. Mixing single and double quotes lets us print a string that itself contains an apostrophe (`'I'll...'`) or double quotes (`"Python is easy..."`) without those characters ending the string prematurely. The raw string prefix `r"..."` (used for the row of backslashes) tells Python to treat backslashes literally instead of as the start of an escape sequence.

Note

[All code for this assignment has been pushed on GitHub, and also shows the result.]